

Understanding and Implementing Authenticated Post-Quantum Key Exchange

A Practical Study of CRYSTALS-Kyber and EKE

Master's degree in Telecommunications and Informatics Engineering

Author:
Inês Martins Ribeiro

Professor:
Ricardo Chaves

Index

1	Introduction	1
2	Background and Foundations	2
2.1	Key Exchange and Authentication	2
2.2	Classical Key Exchange Protocols	2
2.3	Encrypted Key Exchange	3
2.4	Post-Quantum Key Establishment and KEMs	3
2.5	Lattice-Based Cryptography	4
3	CRYSTALS-Kyber	5
3.1	Design Goals and Security Model	5
3.2	Kyber Algorithm Overview	5
3.3	Mathematical Structure of Kyber	5
3.4	Implementation Overview	7
3.5	Key Encapsulation Workflow	8
3.6	Experimental Validation	8
3.7	Results	8
4	Authenticated CRYSTALS-Kyber using EKE	10
4.1	Motivation and Design	10
4.2	Protocol Specification (EKE-Kyber)	10
4.3	Implementation Overview	11
4.4	Experimental Validation	11
4.5	Security Discussion and Limitations	12
5	Conclusion	13

1 Introduction

Over the past decades, classical public-key cryptographic algorithms have played a central role in securing digital communications. Schemes such as RSA and Elliptic Curve Cryptography (ECC) are widely used to provide confidentiality, integrity, and secure key establishment in modern communication protocols. However, due to the recent advancements in the field of quantum computing, these algorithms are now at risk of being compromised by quantum computers.

The development of quantum algorithms, such as Shor's algorithm [1], has demonstrated that quantum computers could efficiently solve problems that are currently considered hard for classical computers, including integer factorization and discrete logarithms. Since the security of RSA, Diffie-Hellman, and ECC relies on the computational hardness of these problems, the advent of large-scale quantum computers would render many existing cryptographic protocols insecure.

In response to this emerging threat, Post-Quantum Cryptography (PQC) has become a major area of research. PQC aims to develop cryptographic algorithms that are resistant to both classical and quantum adversaries. In recent years, several mathematical approaches have been proposed, among which lattice-based cryptography has shown great promise, offering strong security guarantees and practical efficiency.

To address the need for standardized post-quantum solutions, the National Institute of Standards and Technology (NIST) [2] has been leading global efforts to develop and standardize post-quantum cryptographic algorithms. In 2024, NIST announced the first group of standardized post-quantum cryptographic algorithms, which includes CRYSTALS-Kyber [3], a lattice-based Key Encapsulation Mechanism (KEM) that has demonstrated strong security and efficiency properties.

This report focuses on the study and practical implementation of CRYSTALS-Kyber. The goal is to provide a comprehensive overview of Kyber and its role in post-quantum cryptography. Additionally, this work considers the design of an authenticated post-quantum key exchange protocol based on the Encrypted Key Exchange (EKE) framework, utilizing CRYSTALS-Kyber as the underlying KEM to support authentication and security against man-in-the-middle attacks.

2 Background and Foundations

2.1 Key Exchange and Authentication

Key Exchange refers to the process through which two parties establish a shared secret key over an insecure communication channel [4]. These keys are essential for encrypting and decrypting the data being exchanged, ensuring that the information remains confidential and protected from unauthorized access.

While confidentiality is a critical aspect of secure communication, it is not sufficient on its own. In the absence of authentication, communication systems become vulnerable to man-in-the-middle attacks, in which an adversary may not only eavesdrop on the communication but also impersonate one of the parties or actively tamper with the exchanged messages [4].

Authentication is therefore a crucial component of secure communication [4]. It ensures that the parties involved are who they claim to be and that the exchanged messages originate from legitimate participants. As a result, authentication plays a central role in the design of key exchange protocols intended for practical deployment, since without it the established secret key may be shared with an unintended party. A clear understanding of the distinction between key exchange and authentication is essential when analyzing existing protocols and designing secure communication systems.

2.2 Classical Key Exchange Protocols

Classical key exchange mechanisms, such as Diffie-Hellman [5] and RSA-based key establishment schemes, have been widely used to secure communication channels in protocols including TLS, VPNs, and other network security frameworks.

For instance, the Diffie-Hellman key exchange protocol is one of the most well-known and widely adopted solutions. It allows two parties to establish a shared secret over an insecure channel without requiring any prior shared information. The security of Diffie-Hellman relies on the computational hardness of the discrete logarithm problem [5], which is considered infeasible using classical computers. Variants such as Elliptic Curve Diffie-Hellman (ECDH) are extensively used in modern communication protocols due to their efficiency and strong security properties.

Despite their effectiveness against classical adversaries, the security assumptions underlying classical key exchange protocols are challenged by advances in quantum computing. In particular, Shor's algorithm [1] demonstrates that a sufficiently powerful quantum computer can efficiently solve the discrete logarithm problem, rendering Diffie-Hellman and other classical public-key algorithms insecure in a post-quantum setting.

As a result, there is a growing need to reconsider traditional key exchange mechanisms and explore alternative approaches that remain secure against quantum-capable adversaries [2]. This challenge has motivated the development of new cryptographic paradigms for key establishment, which differ significantly from classical designs.

2.3 Encrypted Key Exchange

Encrypted Key Exchange (EKE) is a class of key exchange protocols designed to provide authentication in scenarios where parties share a low-entropy secret, such as a password [6]. Unlike traditional authenticated key exchange protocols that rely on public key infrastructures or digital certificates, EKE enables authentication using a pre-shared secret while preventing common attacks associated with password-based systems.

The main challenge in password-based authentication lies in the limited entropy of passwords, which makes them vulnerable to offline dictionary attacks [6]. EKE addresses this challenge by ensuring that the password is never transmitted directly over the communication channel and is not exposed in a form that allows an adversary to efficiently test password guesses. Instead, the shared password is used to protect cryptographic material exchanged during the protocol execution.

In an EKE-based protocol, the shared password is typically used to encrypt public values exchanged between the communicating parties. As a result, an adversary who does not possess the correct password is unable to meaningfully interact with the protocol or perform man-in-the-middle attacks. Even if the adversary actively intercepts and modifies protocol messages, the lack of knowledge of the password prevents successful impersonation of either party.

An important characteristic of the EKE paradigm is that it is largely independent of the underlying key exchange primitive. While EKE was originally proposed in the context of classical public-key cryptography, its core idea is not tied to a specific mathematical assumption. Instead, EKE provides a general framework for combining authentication based on shared secrets with key exchange mechanisms.

Despite its advantages, EKE also presents inherent limitations. The security of the protocol ultimately depends on the secrecy of the shared password [6], and poor password choices may weaken the overall system. Nevertheless, EKE offers an effective and lightweight authentication mechanism in environments where public key infrastructures are unavailable or impractical, making it a relevant building block for secure key exchange protocol design.

2.4 Post-Quantum Key Establishment and KEMs

Key-Encapsulation Mechanisms (KEMs) have become a central concept in post-quantum cryptography [2], providing a secure and efficient method for establishing a shared secret over an insecure communication channel. Unlike classical key exchange protocols such as Diffie-Hellman, which rely on an interactive key agreement process, KEMs are based on the encapsulation and decapsulation of a secret key.

A KEM typically consists of three main algorithms: key generation, encapsulation, and decapsulation. In the key generation phase, a receiving party generates a public-private key pair, where the public key is distributed and the private key is kept secret. To initiate key establishment, the sending party uses the public key to encapsulate a randomly generated secret, producing a ciphertext. This ciphertext is transmitted over the insecure channel, and upon reception, the receiving party applies the decapsulation algorithm using its private key to recover the shared secret. This secret can then be used as input to symmetric cryptographic primitives for subsequent secure communication.

Compared to Diffie-Hellman, KEMs provide a more structured and straightforward approach to key establishment. As a result, many post-quantum cryptographic protocols adopt KEM-based designs as a replacement for classical key exchange mechanisms. However, a KEM alone does not provide authentication. It inherently assumes that the public key used during the encapsulation phase is authentic, and therefore remains vulnerable to active attacks such as public key substitution and man-in-the-middle attacks.

Despite this limitation, KEMs play a fundamental role in post-quantum cryptography and serve as the foundation for several standardized algorithms. One prominent example is CRYSTALS-Kyber, a lattice-based KEM selected by NIST for post-quantum key establishment [2, 3].

2.5 Lattice-Based Cryptography

Lattice-based cryptography has emerged as one of the leading approaches in post-quantum cryptography [2]. Unlike classical cryptographic schemes that focus on number-theoretic problems, lattice-based cryptography relies on mathematical problems defined over regular and structured grids of points in a high-dimensional space, known as lattices. Because of their mathematical properties, lattice-based schemes are deemed infeasible even for quantum computers, which makes lattice-based approaches strong candidates to replace public-key cryptography in a post-quantum setting.

This cryptographic approach relies on the central hardness assumption of problems such as Learning with Errors (LWE) and the Shortest Vector Problem (SVP) [7]. Essentially, these problems can be described as recovering a hidden secret from a set of multiple equations that have been deliberately tampered with by adding small random errors. While these equations may appear almost correct, the presence of noise makes it computationally difficult to determine the original secret without additional information. As a result, constructing these equations is straightforward when the secret is known. However, recovering the secret from the noisy equations is computationally infeasible [7], even for quantum computers. For this reason, LWE provides a strong foundation for building secure cryptographic schemes in a post-quantum context.

LWE has several variants, such as Ring-LWE and Module-LWE [7], which improve efficiency by introducing additional algebraic structures. This allows for more compact key sizes and faster computations, making lattice-based schemes more practical for real-world applications. Post-quantum solutions often take advantage of these variants to achieve a balance between security and performance. CRYSTALS-Kyber, for example, is a lattice-based KEM built on the Module-LWE [3], which will be further explored in this report.

3 CRYSTALS-Kyber

3.1 Design Goals and Security Model

CRYSTALS-Kyber was selected by NIST as one of the first standardized post-quantum cryptographic algorithms [2, 3]. Kyber is a lattice-based KEM designed to enable secure key establishment in a post-quantum context, allowing two parties to achieve a secret key over an insecure channel without relying on classical key agreement methods. The design of Kyber aims to balance strong security guarantees with practical efficiency requirements, making it suitable for deployment in real-world communication protocols.

From a security perspective, Kyber is based on the hardness of the Module-LWE problem [3]. This algorithm is designed to achieve security against chosen-plaintext attacks and, through standard transformations such as the Fujisaki–Okamoto transform [8], it can be extended to provide resistance against chosen-ciphertext attacks. These security properties, combined with its efficient performance, contributed to Kyber’s selection by NIST as a standardized post-quantum key encapsulation mechanism.

However, it is important to mention that Kyber, as a KEM, does not provide authentication on its own [2]. Therefore, it is essential to integrate Kyber into a broader protocol that includes authentication mechanisms to ensure the integrity and authenticity of the communicating parties.

3.2 Kyber Algorithm Overview

Kyber follows a KEM paradigm [3], which involves three main algorithms: key generation, encapsulation, and decapsulation. During these phases, two parties are able to establish a shared secret over an insecure channel without relying on the security assumptions of classical key exchange methods. Each phase plays a distinct role in the overall operation of the scheme and contributes to its security and correctness.

3.3 Mathematical Structure of Kyber

CRYSTALS-Kyber is a lattice-based KEM whose security relies on the hardness of the Module Learning with Errors (Module-LWE) problem [3]. All mathematical operations in Kyber are defined over a fixed polynomial ring, which provides the structure required to ensure efficiency and security.

Polynomial Ring Definition

Kyber operates over the polynomial ring defined as:

$$R_q = \mathbb{Z}_q[x]/(x^n + 1) \quad (1)$$

where $n = 256$ and $q = 3329$.

Every element of R_q is represented as a polynomial of degree less than n with coefficients reduced modulo q . All polynomial operations such as addition and multiplication are performed in this ring so that all results remain within the defined structure.

All values in Kyber, including keys, ciphertexts, error terms and other components are represented as polynomials in R_q .

Module-LWE Assumption

The security of Kyber is based on the assumption that given a public matrix $A \in R_q^{k \times k}$ and a vector

$$\mathbf{t} = A\mathbf{s} + \mathbf{e} \quad (\text{mod } q) \quad (2)$$

it is computationally infeasible to recover the secret $\mathbf{s} \in R_q^k$ because of the presence of the error $\mathbf{e} \in R_q^k$. This error, also referred to as noise, is intentionally sampled from a distribution centered around zero and added to the equations in order to prevent efficient recovery of the secret by an adversary.

Key Generation

In order to establish a communication channel, the receiving party, Bob, generates a small secret $\mathbf{s}$ and an error $\mathbf{e}$. Using a publicly generated matrix A , Bob computes:

$$\mathbf{t} = A\mathbf{s} + \mathbf{e} \quad (\text{mod } q). \quad (3)$$

and sends to Alice his public key $pk = (\text{seed}, A, \mathbf{t})$. The seed allows Alice to deterministically reconstruct the same matrix A while t contains the secret s with added noise. Bob keeps his secret key, which includes the vector $\mathbf{s}$, private.

Encapsulation

Alice upon receiving Bob's public key, generates her own small secret $\mathbf{r}$ and error terms $\mathbf{e}_1$ and $\mathbf{e}_2$. She also reconstructs the matrix A from the seed included in pk . Using these values, Alice computes the ciphertext components as follows:

$$\mathbf{u} = A^T \cdot \mathbf{r} + \mathbf{e}_1 \quad (\text{mod } q) \quad (4)$$

$$v = \mathbf{t}^T \mathbf{r} + e_2 + \text{enc}(m) \quad (\text{mod } q) \quad (5)$$

where $\text{enc}(m)$ is the encoding of a randomly generated message m . The ciphertext is then formed as $ct = (\mathbf{u}, v)$ and sent to Bob. Alice also derives the shared secret ss from the message m using a Key Derivation Function (KDF).

Decapsulation

Upon receiving the ciphertext $ct = (\mathbf{u}, v)$, Bob performs an algebraic operation to recover the encoded message. Using his secret key $\mathbf{s}$, Bob computes

$$w = v - \mathbf{s}^T \mathbf{u} \pmod{q}. \quad (6)$$

Recall that the ciphertext components were generated as

$$\mathbf{u} = \mathbf{A}^T \mathbf{r} + \mathbf{e}_1 \pmod{q}, \quad (4)$$

$$v = \mathbf{t}^T \mathbf{r} + e_2 + \text{enc}(m) \pmod{q}, \quad (5)$$

where the public key component $\mathbf{t}$ is defined as

$$\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e} \pmod{q} \quad (3)$$

Substituting $\mathbf{t}$ and $\mathbf{u}$ into the expression for w , we obtain

$$w = (\mathbf{A}\mathbf{s} + \mathbf{e})^T \mathbf{r} + e_2 + \text{enc}(m) - \mathbf{s}^T (\mathbf{A}^T \mathbf{r} + \mathbf{e}_1) \pmod{q} \quad (7)$$

$$w = \text{enc}(m) + \mathbf{e}^T \mathbf{r} + e_2 - \mathbf{s}^T \mathbf{e}_1 \pmod{q}. \quad (8)$$

The secret-dependent term $\mathbf{s}^T \mathbf{A}^T \mathbf{r}$ cancels out, leaving the encoded message with some noise. Since this noise is sufficiently small, Bob can correctly decode the message m with overwhelming probability.

After recovering the message m , Bob derives the shared secret ss using the same Key Derivation Function (KDF) as Alice.

3.4 Implementation Overview

This section describes the implementation setup used to verify the functional correctness of the CRYSTALS-Kyber KEM. The implementation is based on the official reference code provided by the PQ-Crystals project [9] and was used without any modifications to ensure compliance with the original specifications.

Since the primary goal of this implementation is to experimentally verify and validate the Kyber algorithm, the reference code located in the `ref/` was selected due to its clarity. In addition, optimized versions of the code are also available in the `avx2/` directory, but they were not used at this stage.

All tests were conducted in C, as the reference code is written in the same language, and were compiled using the GCC compiler in a Linux environment.

3.5 Key Encapsulation Workflow

To experimentally validate Kyber, a simple two-party communication model was implemented. The model consists of a sender, Alice, and a receiver, Bob. The goal is for both parties to establish a shared secret using Kyber encapsulation and decapsulation processes.

In the first step, Bob generates a public-private key pair using the Kyber key generation algorithm. The public key is shared with Alice while the private key remains secret and is used during the decapsulation phase.

Alice receives Bob's public key and executes the encapsulation algorithm to generate the ciphertext and the shared secret. The ciphertext is sent to Bob while the shared secret is kept locally by Alice.

After receiving the ciphertext from Alice, Bob uses his private key to decapsulate the ciphertext and retrieve the shared secret.

Finally, both parties compare their derived shared secrets to verify that they match, confirming the correctness of the Kyber implementation.

3.6 Experimental Validation

A series of tests were conducted to validate the correctness of the Kyber implementation. As previously stated, an execution is considered successful if both parties achieve the same shared secret after all KEM phases.

As a first step, the official tests provided in the reference code were executed. These tests cover various levels of security: Kyber-512, Kyber-768, and Kyber-1024. Each security level was tested multiple times to ensure consistency. Additional checks were also performed using altered ciphertexts, reflecting a chosen-ciphertext attack scenario.

Beyond the official tests, a custom standalone Alice-Bob demo was implemented following the description in the previous subsection. This demo was executed multiple times for each security level to further validate the implementation.

Together, these tests aimed to provide a systematic validation of the correctness of the CRYSTALS-Kyber KEM implementation.

3.7 Results

All experimental tests described in the previous subsection were successfully completed with no failures observed. The following table summarizes the results obtained for each security level of Kyber:

Kyber Variant	Public Key (bytes)	Secret Key (bytes)	Ciphertext (bytes)	Result
Kyber512	800	1632	768	Passed
Kyber768	1184	2400	1088	Passed
Kyber1024	1568	3168	1568	Passed

Table 1: Experimental results for the CRYSTALS-Kyber key encapsulation mechanism

As expected, the increase of security levels resulted in larger key sizes and ciphertexts. This shows the trade-off between security and performance. Achieving higher security requires larger keys and ciphertexts, which may impact performance in resource-constrained environments.

In addition to parameter-level observations, the table below summarizes the coverage and outcomes of the experimental validation procedures:

Test Type	Kyber512	Kyber768	Kyber1024
Official KEM Tests (<code>test_kyber*</code>)	Passed	Passed	Passed
CCA-style Ciphertext Checks	Passed	Passed	Passed
Reference Test Vectors (<code>test_vectors*</code>)	Passed	Passed	Passed
Alice–Bob Demonstration	Passed	Passed	Passed

Table 2: Coverage and outcomes of the experimental validation procedures

The successful execution of the official KEM tests, the validation against reference test vectors, the handling of modified ciphertexts, and the standalone Alice–Bob demonstration collectively provide strong evidence of the correctness and robustness of the implemented system.

4 Authenticated CRYSTALS-Kyber using EKE

4.1 Motivation and Design

While CRYSTALS-Kyber provides an efficient mechanism for key establishment, it does not offer authentication on its own [3]. It assumes that the public key used during encapsulation is authentic, an assumption that may not hold in practical deployment scenarios. As a result, the lack of authentication makes the protocol vulnerable to man-in-the-middle attacks. Therefore, it is essential to combine Kyber with an authentication mechanism to ensure the integrity and authenticity of the communicating parties.

In this context, password-based authentication can be an effective solution, particularly in environments where public key infrastructures are unavailable or impractical. The Encrypted Key Exchange (EKE) paradigm [6] provides a framework for integrating password-based authentication with key exchange mechanisms. By using a shared password to protect the key exchange process, EKE prevents active attacks and ensures that only parties who know the correct password can successfully establish a shared secret.

The design presented in this work is based on the classical EKE–Diffie–Hellman protocol [6] and adapts its core principles to use CRYSTALS-Kyber as the underlying Key Encapsulation Mechanism. By integrating Kyber into an EKE-style construction, the proposed approach aims to achieve authenticated key establishment while preserving resistance against quantum-capable adversaries.

4.2 Protocol Specification (EKE-Kyber)

The proposed protocol follows the Encrypted Key Exchange (EKE) paradigm while using CRYSTALS-Kyber as the underlying post-quantum Key Encapsulation Mechanism. The protocol involves two parties, Alice and Bob, who share a previously established password pw . This password is assumed to be known only to the legitimate parties and is never transmitted in clear over the communication channel.

The protocol begins with Bob generating a random salt and deriving a symmetric key from the shared password using a password-based key derivation function. Specifically, a password-derived key is computed as

$$pw_{Bob} = \text{PBKDF2}(pw, \text{salt}).$$

This key is used exclusively to protect the exchange of public key material and is independent of the final shared secret.

Bob then generates a CRYSTALS-Kyber key pair (pk_{Bob}, sk_{Bob}) . To ensure authenticity, Bob encrypts and authenticates his Kyber public key pk_{Bob} using the derived key pw_{Bob} and the AES-GCM authenticated encryption scheme. The resulting ciphertext ct_{pk} , together with the salt, initialization vector iv_{pk} , and authentication tag tag_{pk} , is sent to Alice.

Upon receiving these values, Alice derives the same password-based key pw_{Alice} using the shared password and the received salt. Alice then decrypts and verifies the ciphertext ct_{pk} using AES-GCM with the received parameters. If the decryption or authentication fails, the protocol is aborted. Otherwise, Alice successfully recovers Bob’s Kyber public key pk_{Bob} .

After obtaining a valid public key, Alice executes the Kyber encapsulation algorithm using pk_{Bob} , producing a ciphertext ct_K and a shared secret ss_A . The ciphertext ct_K is then sent to Bob over the insecure channel.

Bob receives the ciphertext ct_K and performs the Kyber decapsulation algorithm using his private key sk_{Bob} , recovering a shared secret ss_B . Correctness is verified by checking that $ss_A = ss_B$.

At the end of this phase, Alice and Bob share a common post-quantum secret. Authentication is achieved implicitly, since only a party possessing the correct password is able to decrypt the protected public key and successfully complete the encapsulation and decapsulation process.

4.3 Implementation Overview

The implementation of the EKE-Kyber protocol was designed as a minimal and transparent proof of concept. The objective was to demonstrate how authentication can be added to a post-quantum key exchange protocol using the Encrypted Key Exchange (EKE) paradigm.

The program was developed in the C programming language, using the OpenSSL library to implement standard cryptographic primitives such as AES-256-GCM for authenticated encryption and PBKDF2 for password-based key derivation.

Post-quantum key establishment was implemented using the official CRYSTALS-Kyber reference code provided by the PQ-Crystals project. This code was used without any modifications in order to ensure full compliance with the original specifications.

The protocol logic is implemented in a single standalone program that simulates the interaction between two parties, Alice and Bob. The implementation focuses on clarity and functional correctness rather than performance optimizations or resistance to side-channel attacks.

4.4 Experimental Validation

The experimental validation of the EKE-Kyber protocol focuses on verifying that both parties are able to successfully establish a shared secret while ensuring that authentication is properly enforced.

The implementation was executed multiple times to confirm that the derived password-based keys at Alice and Bob are identical when using the same password and salt. This validates the correct use of the PBKDF2 key derivation function and ensures that both parties operate under the same symmetric key during the authentication phase.

In addition, the program was tested using Kyber's three security levels (Kyber-512, Kyber-768, and Kyber-1024) to confirm that the protocol functions correctly across different parameter sets. In all cases, the protocol successfully established a shared secret between Alice and Bob, confirming the correctness of the implementation.

To further validate the authentication mechanism, tests were conducted with incorrect passwords. In these scenarios, the decryption of Bob's public key by Alice failed as expected, and the protocol was correctly aborted, demonstrating that authentication is effectively enforced.

Overall, the experimental validation confirms that the EKE-Kyber protocol successfully integrates authentication with post-quantum key establishment, providing a secure mechanism for two parties to establish a shared secret over an insecure channel.

4.5 Security Discussion and Limitations

The EKE-Kyber protocol provides a secure mechanism for authenticated key establishment in a post-quantum context. By protecting the public key exchange with a password-based authentication mechanism, the protocol provides resistance against man-in-the-middle attacks while leveraging the security properties of CRYSTALS-Kyber to ensure the confidentiality of the established shared secret.

Despite its advantages, the protocol also presents several limitations. In particular, the overall security relies heavily on the secrecy and strength of the shared password. If the password is weak or compromised, an adversary may be able to impersonate one of the parties and successfully interfere with the key establishment process. Although the use of PBKDF2 with a salt and a high iteration count mitigates the risk of offline dictionary attacks, it does not eliminate the inherent vulnerabilities associated with password-based authentication.

Therefore, it is crucial to encourage the use of strong, high-entropy passwords and to consider additional security measures to enhance the overall robustness of the system. Furthermore, the current implementation focuses on functional correctness and does not address side-channel attacks or other implementation-specific vulnerabilities, which would need to be carefully considered in a production environment.

5 Conclusion

This report provided an overview of the NIST-standardized post-quantum key encapsulation mechanism CRYSTALS-Kyber, covering its design goals, mathematical structure, and practical implementation. Experimental validation confirmed the functional correctness of the Kyber reference implementation across different security levels, demonstrating that both parties consistently derive the same shared secret after encapsulation and decapsulation.

In addition, this work proposed an authenticated post-quantum key establishment approach based on the Encrypted Key Exchange (EKE) paradigm, integrating CRYSTALS-Kyber to address the lack of authentication inherent to standalone KEM deployments. The implemented EKE-Kyber proof of concept showed that a shared secret can be established while implicitly enforcing authentication by protecting the exchange of Kyber public key material with password-derived symmetric encryption. The validation also confirmed that protocol execution is aborted when incorrect password material is used, as expected.

Despite these results, the proposed design inherits the limitations of password-based authentication, making overall security dependent on password secrecy and strength. For this reason, strong password selection and appropriate password-hardening mechanisms remain essential. Furthermore, the current implementation focuses on clarity and functional correctness and does not address side-channel resistance or provide a formal security proof under a standard authenticated key exchange model.

Overall, this work contributes to a better understanding of post-quantum key establishment and highlights the importance of integrating authentication mechanisms when designing practical post-quantum secure communication protocols. The EKE-Kyber construction presented serves as a clear and minimal example of how authentication can be introduced using standard cryptographic primitives combined with a post-quantum KEM. The source code for the EKE-Kyber implementation developed in this study is publicly available on GitHub [10].

References

- [1] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, 1994.
- [2] National Institute of Standards and Technology. Post-quantum cryptography standardization. <https://csrc.nist.gov/projects/post-quantum-cryptography>, 2024. Accessed 2026.
- [3] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber algorithm specifications and supporting documentation. <https://pq-crystals.org/kyber/data/kyber-specification-round3.pdf>, 2021. Accessed 2026.
- [4] Alfred J. Menezes, Scott A. Vanstone, and Paul C. van Oorschot. Authenticated key exchange protocols. In *Advances in Cryptology – CRYPTO’96*, 1996. Available online: <https://www.cs.columbia.edu/~smb/papers/neke.pdf>.
- [5] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.
- [6] Steven M. Bellovin and Michael Merritt. Encrypted key exchange: Password-based protocols secure against dictionary attacks. In *Proceedings of the IEEE Symposium on Security and Privacy*, pages 72–84, 1992.
- [7] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *Proceedings of the 37th Annual ACM Symposium on Theory of Computing (STOC)*, pages 84–93, 2005.
- [8] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *Advances in Cryptology – CRYPTO’99*, pages 537–554. Springer, 1999.
- [9] PQ-Crystals Team. Crystals-kyber reference implementation (repository). <https://github.com/pq-crystals/kyber>. Accessed 2026.
- [10] Inês Martins Ribeiro. Eke-kyber proof-of-concept implementation. <https://github.com/InesMRibeiro/pqe-eke-kyber>, 2026. Accessed: 2026.